TOPIC 3

DESIGN A SOLUTION

COMPUTER SCIENCE II| SC025

Prepared by: Ms Jasini



Content

3.0 Introduction to algortihms (pseudocode & flowcharts)

- Sequence Control Structures
- Selection Control Structures
- Repetition Control Structures
- Tracing Outputs from algorithms

Learning Outcome

At the end of the lesson, students should be able to:

a) **Describe** algorithm: **pseudocode** and **flowchart**.

b) **Explain control structures** (sequence, selection : single, dual, multiple, repetition: counter- controlled, sentinel-controlled) **using algorithms**.

c) **Apply** appropriate control structures in **computational problem solving**.

Introduction

Design a solution involves **creating** **algorithm** (pseudocode or flowchart) that is a **series of well-defined steps** to **solve a problem.**

Qualities of good algorithm:

- Input and output values **defined precisely** (accurately).
- **Each step** in the algorithm should be **clear and general**.
- The chosen algorithm should be the **most effective** to solve the given problem.
- The algorithm should be written in a way so it **can be used** for **different programming languages** for implementation.
- It must execute its steps and **terminate in finite time** (not go on forever).

Pseudocode

Pseudocode is an **informal, English-like language** with **limited vocabulary** that is used to **describe** the **steps** in an **algorithm**.

Advantages:

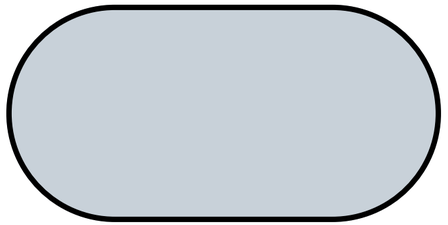
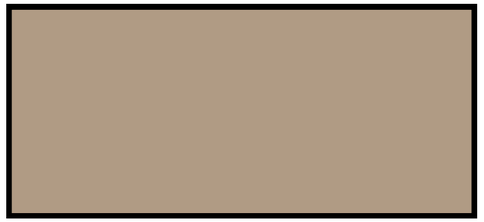
- **Easy to understand**, even to non-programmers.
- Programmers can **focus on** main **logic** without worrying about programming language syntax (rules & structure).
- Works as rough documentation.
- **Explains in detail** what each line of program should do.

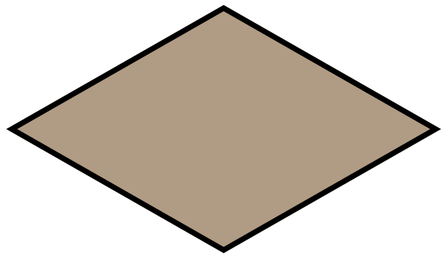
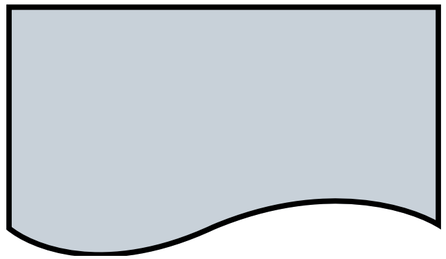
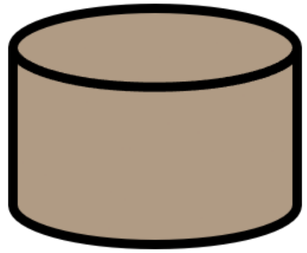
Flowchart

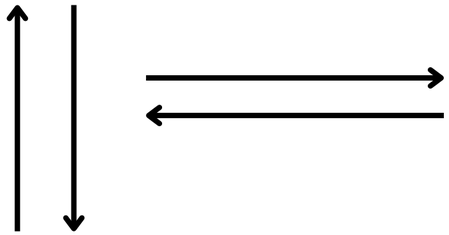
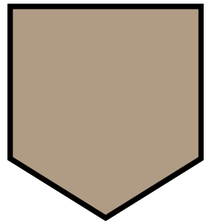
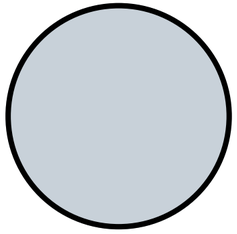
Flowchart is a **graphical representation** of an **algorithm** which uses **shapes** that are **connected by flow lines** to show **flow of statement execution**.

Advantages:

- **Effective way of communicating** the **logic** within a system (visual representation of data).
- Flowcharts are important for **good program documentation**.
- Flowcharts **act as a blueprint** during the analysis and development phase.

Description	Pseudocode Statement	Flowchart Symbol	Symbol Name
Indicates the start and end points of an algorithm	**START / BEGIN** **STOP / END**		Terminal symbol
Processes besides input, output, selections, and repetitions	**INITIALIZE / SET** **CALCULATE**		Process symbol
Shows input and output operations	**READ / INPUT** **PRINT / DISPLAY**		Input-Output symbol

Description	Pseudocode Statement	Flowchart Symbol	Symbol Name
To show a condition	**IF** **WHILE**		Decision symbol
Shows physical hardcopy printer output			Print output
Indicates input from or output to disk storage			Disk-storage I/O

Description	Pseudocode Statement	Flowchart Symbol	Symbol Name
To show the logical flow of execution in the algorithm			Flow lines
To show continuation of flowchart on another page			OFF-page connector
To show continuation of flowchart on the same page			ON-page connector

Introduction to Programming Terms

Variable

- A variable is used to store values in programming.
- It has a name (identifier).
- Rules for variable names:
  - Does not contain whitespace i.e. “ ”
  - Should not begin with uppercase letters or numbers.
  - Cannot be *reserved words (keywords)*
  - Symbols other than _ and $ are not allowed.
- Good examples: *num1, x, y, studentAge, time_minutes*
- Bad examples: *1num, Sum, true, my age, em@il*

Introduction to Programming Terms

Initialization

- Initialization is the process of setting initial values to variables before use. Without this step, there will be errors when trying to use a variable without a value.
- Use keyword *set* in algorithms (pseudocode & flowchart)

```
...
Read num1
sum = sum + num1
Read num2
...
```



```
...
Set sum = 0
Read num1
sum = sum + num1
Read num2
...
```



Sequence Control Structure

Pseudocodes & Flowcharts

Sequence Control Structure

Sequence control structure is a **series of actions** that are **sequentially executed** in the order they are written in an **algorithm**.

- There are no conditions.
- Performs actions **one after another** in a sequence.
- **All actions will be executed**, none will be skipped.
- Process keywords: **calculate, compute, count, add, total, sum, ...**

Example 1

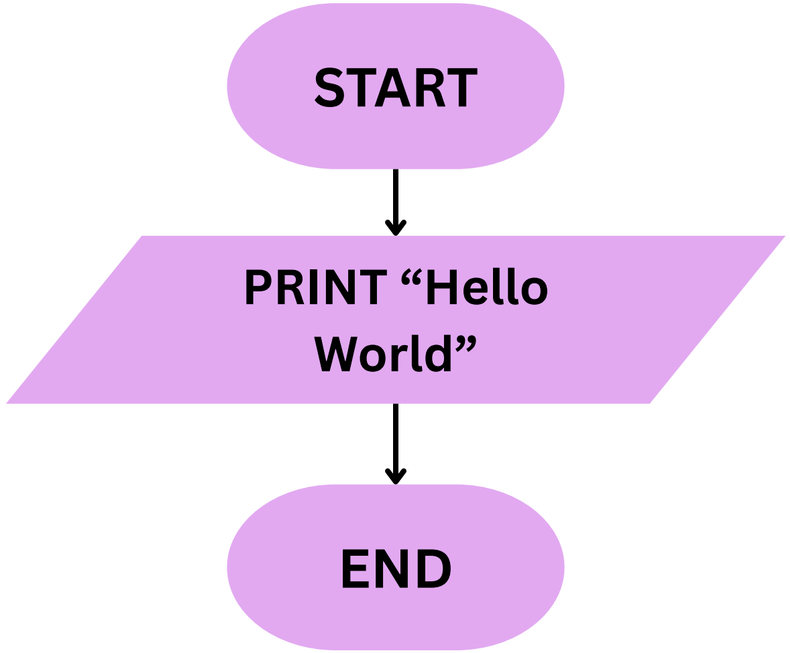
Print the message “Hello World”.

INPUT	PROCESS	OUTPUT
none	none	“Hello World”

Pseudocode:

```
START
    PRINT “Hello World”
END
```

Flowchart:



Example 2

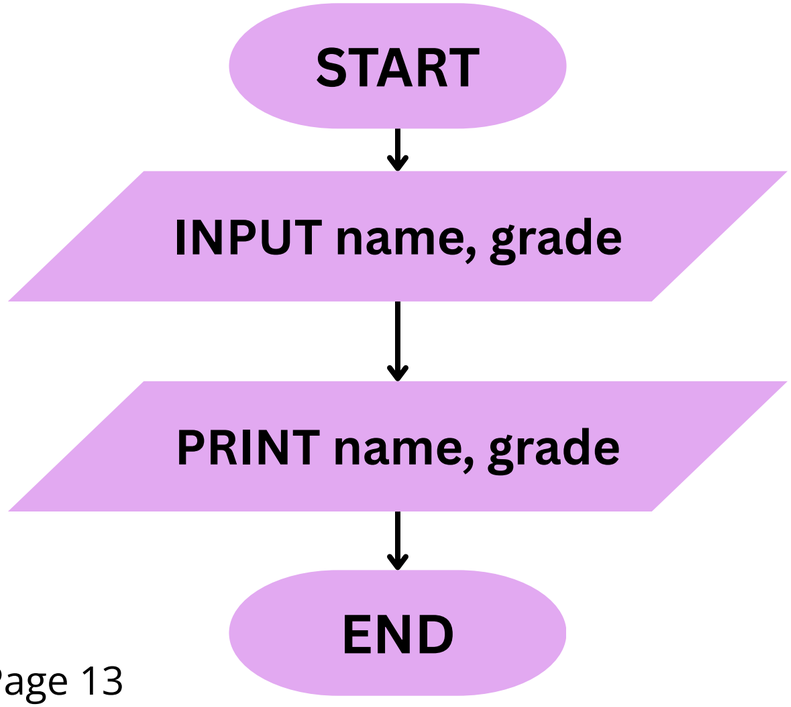
Display the name and grade entered by the user.

INPUT	PROCESS	OUTPUT
name, grade	none	name, grade

Pseudocode:

```
START
    INPUT name, grade
    PRINT name, grade
END
```

Flowchart:



Example 3

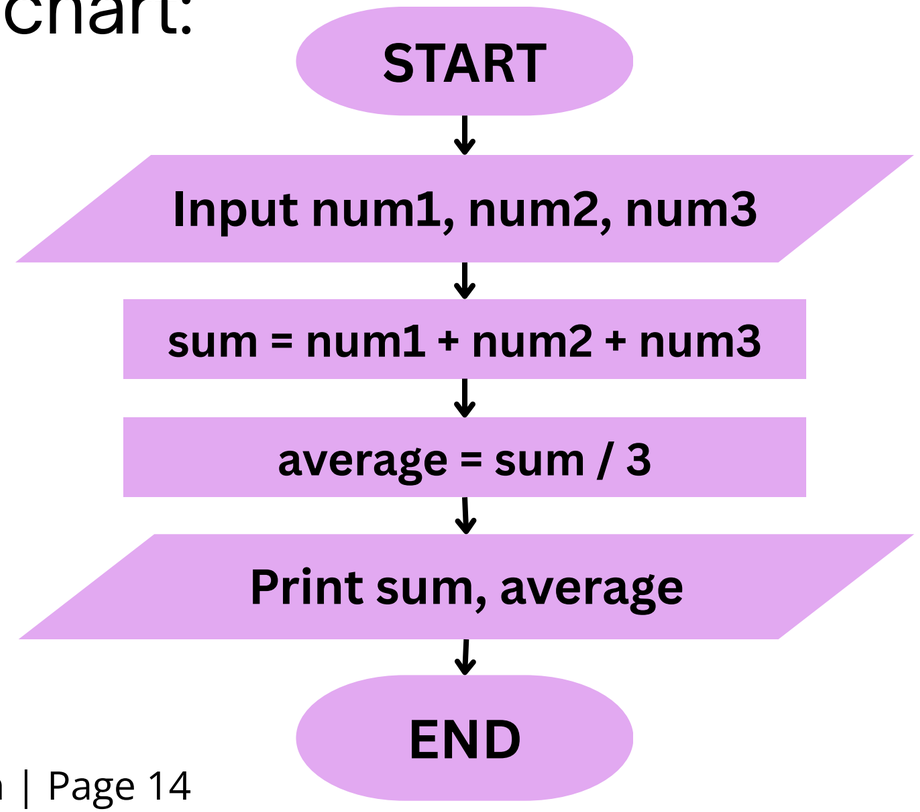
Calculate and display the sum and average of three numbers.

INPUT	PROCESS	OUTPUT
three numbers	calculate sum and average based on the three numbers	sum, average

Pseudocode:

```
START
    INPUT num1, num2, num3
    sum = num1 + num2 + num3
    average = sum / 3
    PRINT sum, average
END
```

Flowchart:



Practice Questions

Do problem analysis, then design the solution (pseudocode & flowchart) for the problem statement(s) below.

1. Final mark for a student is calculated from the total of 60% of the continuous evaluation mark and 40% final examination mark. Calculate and print the final mark for a student.

2. A program that determines the area of a circle with radius *r*.

Selection Control Structure

Pseudocodes & Flowcharts

Selection Control Structure

Selection control structure performs actions based on a **certain conditions**.

- Outputs from selection control structures depends on conditions.
- There has to be **two or more outputs**. Different conditions produce different outputs.
- Process Keywords: **if, else, otherwise, or, based on, depends, determine, find, check, compare**.
- There are three (3) types of selection control structure: **single selection**, **dual selection**, **multiple selection**.

Selection Control Structure

- **Conditions** are ***boolean*** (**true/false**) **expressions** that **determines** the following **statement to be executed** based on its evaluation.
- In **pseudocode**, selection is represented by using ***if***, ***if-else***, or **if-else-if** statements.
- In **flowcharts**, selection is represented by using the **selection symbol**.
- There are **three (3) types** of **selection control structures**:
  - Single selection (*if* statements)
  - Dual selection (*if-else* statements)
  - Multiple selections (*if-else-if* statements)

Example 1 (Single selection)

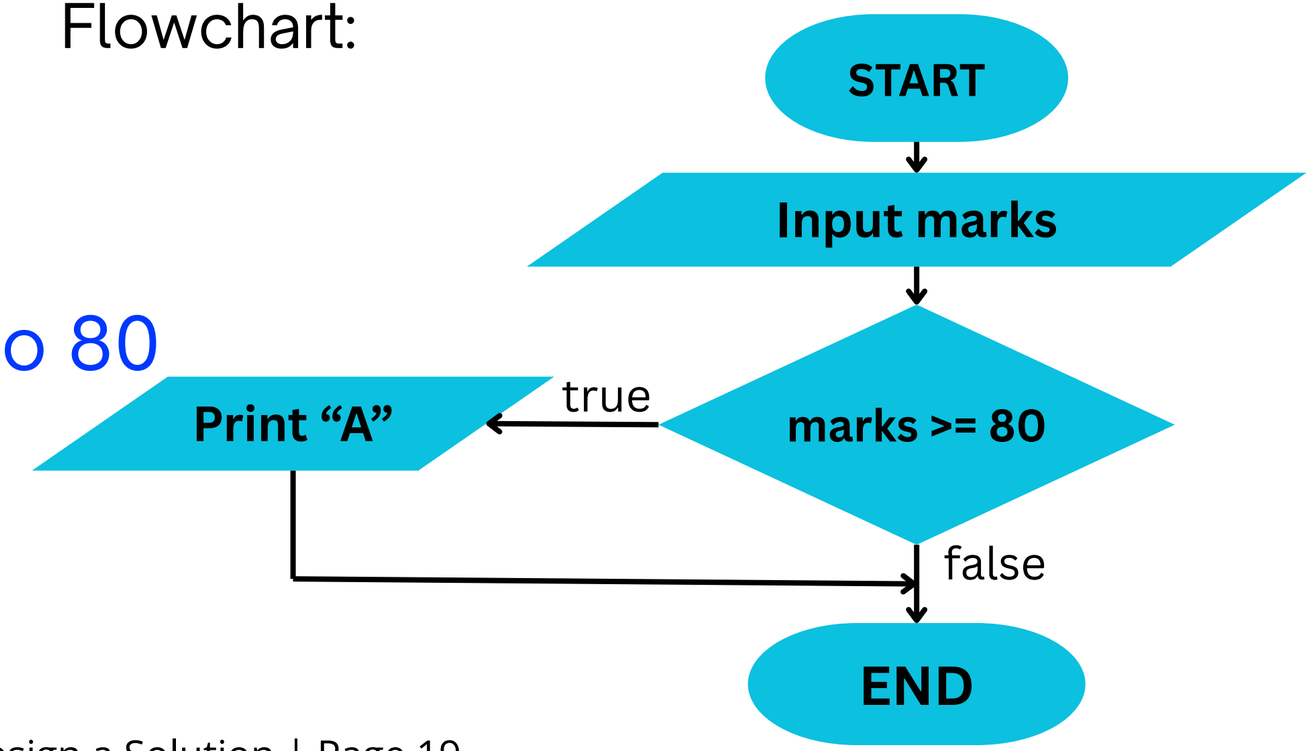
The program will display “A” if mark entered is greater or equal to 80.

INPUT	PROCESS	OUTPUT
marks	determine message to display based on whether marks greater than or equal to 80	“A” or null / no output

Pseudocode:

```
START
    INPUT marks
    IF marks greater or equal to 80
        PRINT “A”
    ENDIF
END
```

Flowchart:



Example 2 (Dual selection)

Determine if a number is even or odd then display “even” or “odd”.

INPUT	PROCESS	OUTPUT
number	determine the message to display based on whether the number is even or odd	“even” or “odd”

Pseudocode:

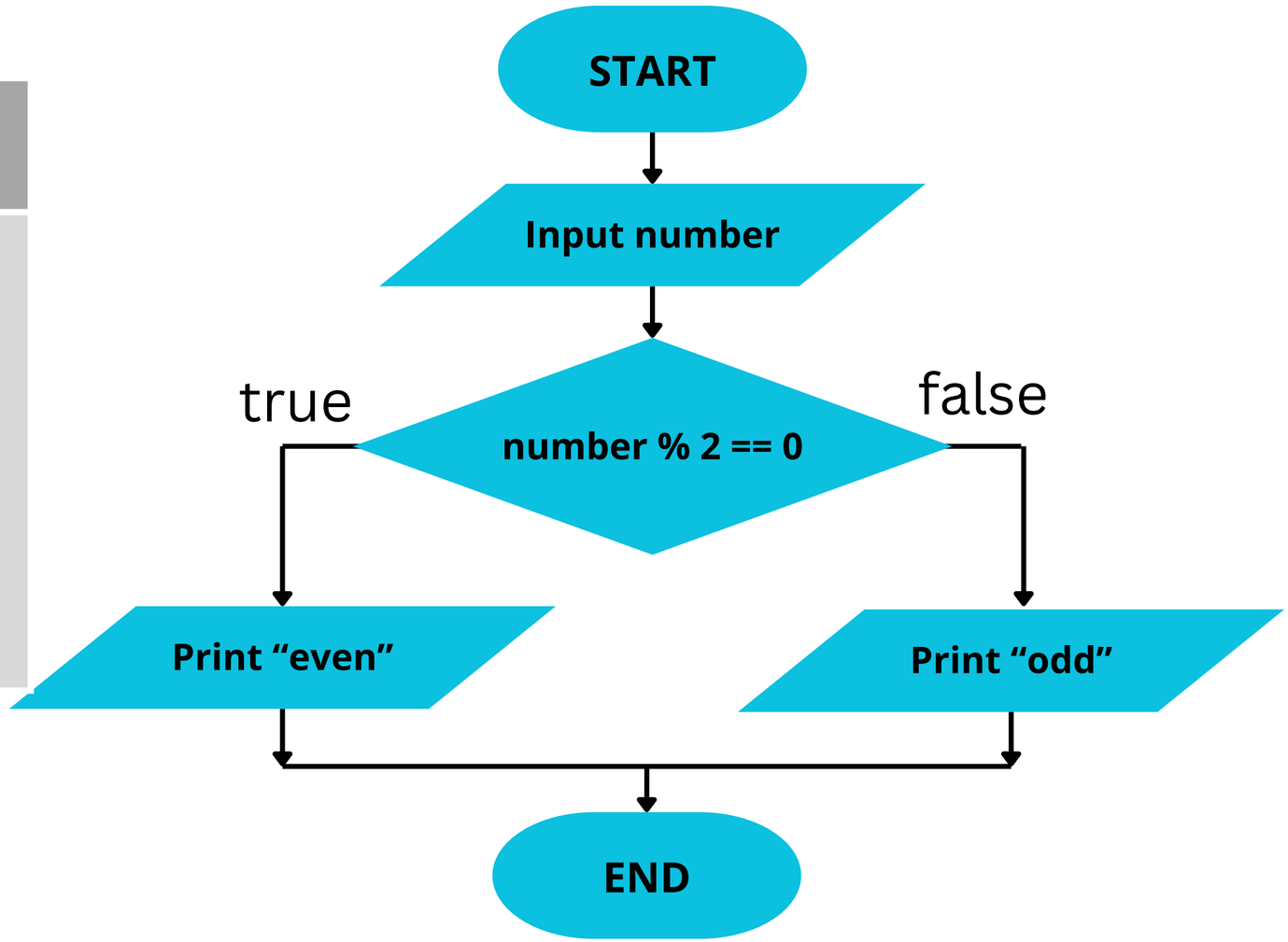
```
START
    INPUT number
    IF number divisible by 2
        PRINT “even”
    ELSE
        PRINT “odd”
    ENDIF
END
```

Example 2 (Dual selection)

Determine if a number is even or odd then display “even” or “odd”.

INPUT	PROCESS	OUTPUT
number	determine the message to display based on whether the number is even or odd	“even” or “odd”

Flowchart:



Example 3 (Multiple selection)

Check whether a number is positive, negative or zero.

INPUT	PROCESS	OUTPUT
number	determine whether the input number is positive, negative, or zero	“positive” or “negative” or “zero”

***If question does not specify display output, you MUST make your own (make sure it makes sense)**

Pseudocode:

```
START
    INPUT num
    IF num greater than 0
        PRINT “positive”
    ELSE IF num less than 0
        PRINT “negative”
    ELSE
        PRINT “zero”
    ENDIF
END
```

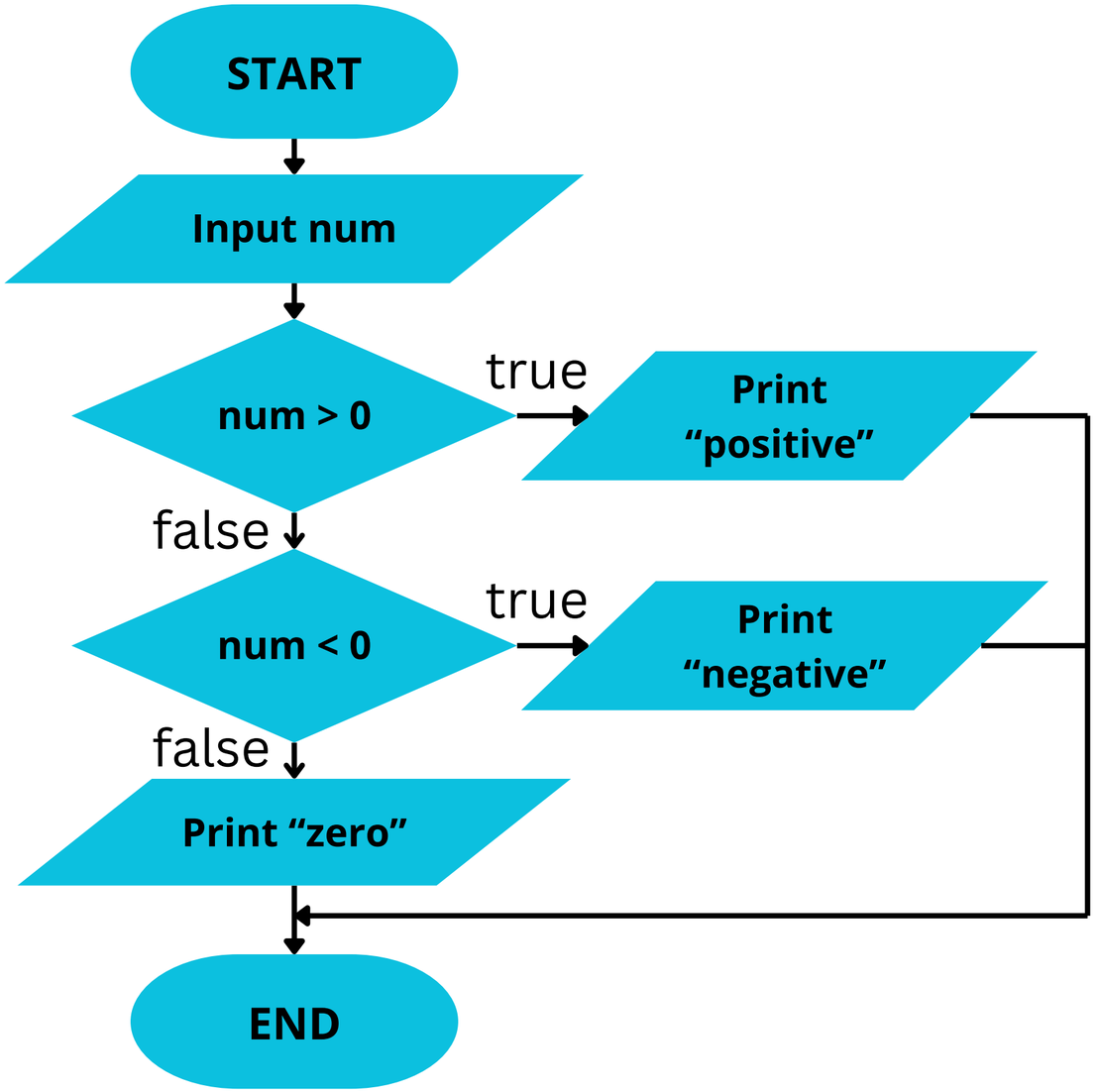
Example 3 (Multiple selection)

Check whether a number is positive, negative or zero.

INPUT	PROCESS	OUTPUT
number	determine whether the input number is positive, negative, or zero	“positive” or “negative” or “zero”

***If question does not specify display output, you MUST make your own (make sure it makes sense)**

Flowchart:



Practice Questions

Do problem analysis, then design the solution (pseudocode & flowchart) for the problem statement(s) below.

1. The program will display “OVERHEAT!!” if car temperature entered is greater or equal to 80°C or “NORMAL” if car temperature between 70- 79°C or “COOL” if temperature entered is below 70°C.

2. Program to display price of ticket based on age: RM2 for a child (no more than 11 years), RM3 for a senior (at least 65 years), or RM5 for an adult.

Repetition Control Structure

Pseudocodes & Flowcharts

Repetition Control Structure

Repetition control structure specifies a block of **one or more statements** that are **repeatedly executed** until a **condition is satisfied**.

- It is **used when** there are **many inputs**, **outputs**, or **processes** to be **repeated**.
- In **pseudocode**, the keyword ***while*** is used.
- In **flowcharts**, the **decision symbol** is used along with an arrow that **loops** back.
- Keyword: **repeat, loop, until, while, as long as, among.**
- There are two (2) types of repetition loop controls: **counter-controlled loop** and **sentinel-controlled loop**.

Repetition Control Structure

Repetition control structure involves **three (3) basic processes**:

1. **Initialization**
   - Set the initial value of the loop control variable (counter OR sentinel value)
2. **Condition / Evaluation**
   - Check the loop control variable to determine whether the loop should continue or not.
3. **Update**
   - Increment / Decrement the value of the loop control variable.

Counter-controlled Loop

- In a **counter-controlled loop**, the **number of repetitions** needed is **defined** (a fixed value).
- The **loop execution** depends on either the:
  - number of inputs to process,
  - number of outputs to display.
- **Example:**
  - Display the message “Hello, world!” 5 times.
  - Read 15 numbers then calculate the sum.
  - Ask user the number of items they bought then ask user the price of each item and display the total at the end.

Example 1 (Counter-controlled)

Print numbers from 1 to 10.

INPUT	PROCESS	OUTPUT
(none)	loop through and print numbers 1 to 10	1 2 3 4 5 6 7 8 9 10

Pseudocode:

```
START
    SET counter = 1
    WHILE counter less than equal to 10
        PRINT counter
        counter = counter + 1
    END WHILE
END
```

Initialization

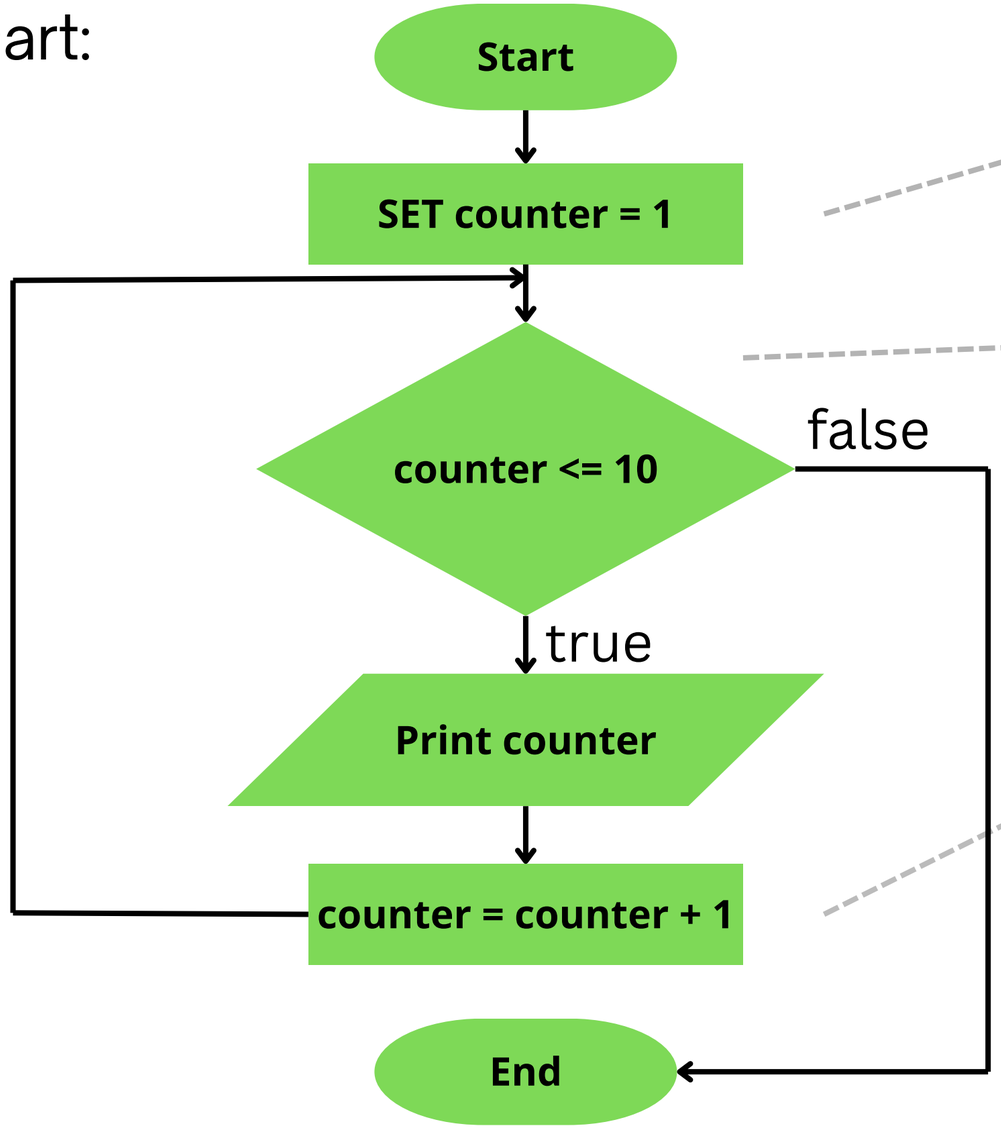
Condition

Update

Example 1 (Counter-controlled)

Print numbers from 1 to 10.

Flowchart:



Initialization

Condition

Update

Example 2 (Counter-controlled)

Program to find the sum and average of three numbers.

INPUT	PROCESS	OUTPUT
number	loop input and calculate sum until three numbers entered, calculate average of the three numbers	sum, average

Pseudocode:

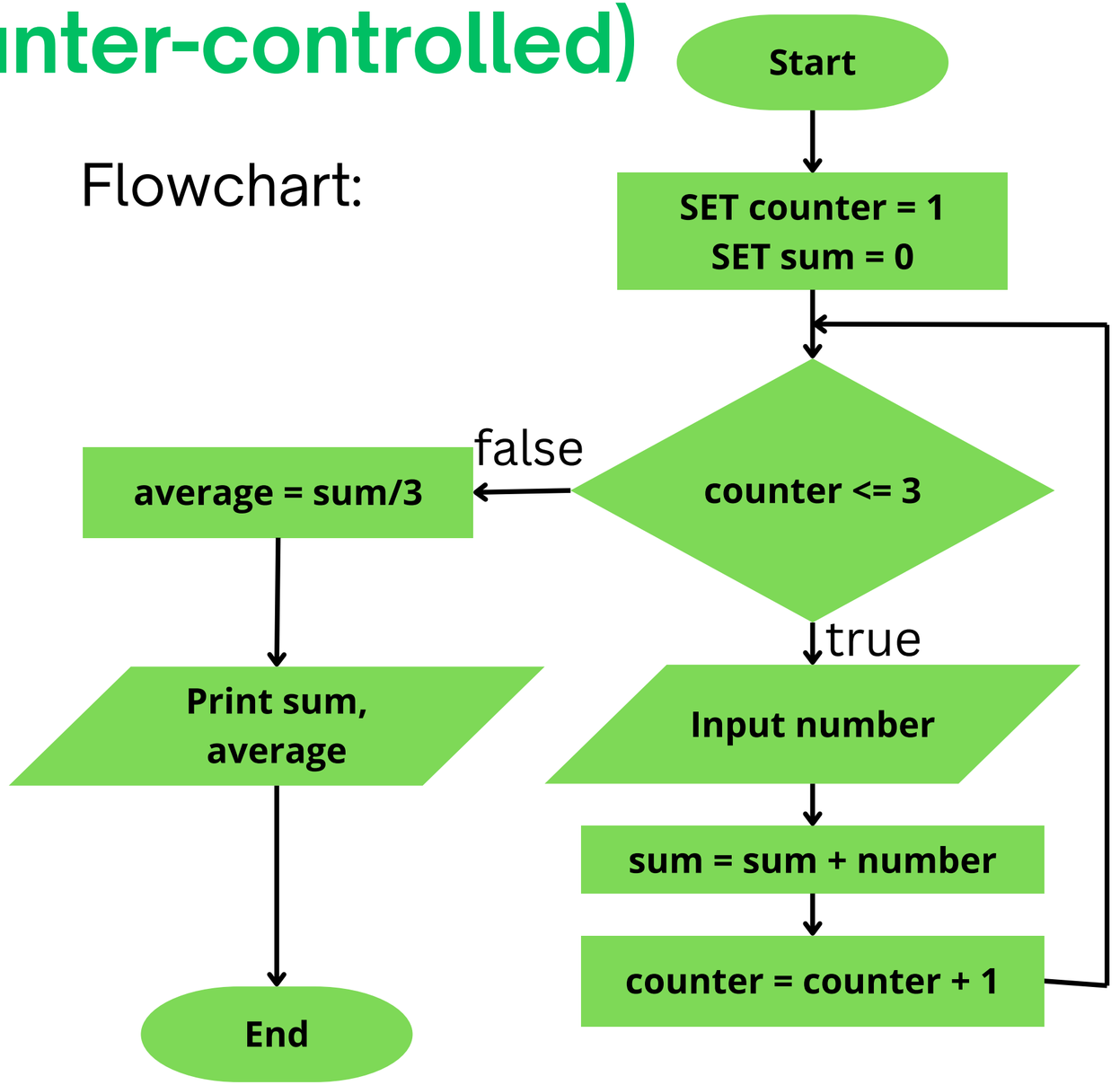
```
START
    SET counter = 1
    SET sum = 0
    WHILE counter less than equal to 3
        INPUT number
        sum = sum + number
        counter = counter + 1
    END WHILE
    average = sum/3
    PRINT sum, average
END
```

Example 2 (Counter-controlled)

Program to find the sum and average of three numbers.

INPUT	PROCESS	OUTPUT
number	loop input and calculate sum until numbers entered, calculate average of the three numbers	sum, average

Flowchart:



Practice Questions

Do problem analysis, then design the solution (pseudocode & flowchart) for the problem statement(s) below.

1. A program reads 20 positive numbers and calculates their sum. It displays the sum after reading 20 positive numbers.

2. Madam Punita has 15 students. For each student, she enters a grade, if the grade is below 40 print “FAIL”, otherwise print “PASS”.

Sentinel-controlled Loop

- In a **sentinel-controlled loop**, the **number of repetitions** is **undefined** or **depends on a condition**.
- A **sentinel value** is used to **signal** when the **loop should stop** (termination condition).
- **Example:**
  - Read a series of numbers until **-1** is read as input.
  - Calculate the total of a few numbers until number **0** is entered by user.
  - Display the even number entered by user, program will stop if a **negative value** is entered.

Example 3 (Sentinel-controlled)

Calculate the total of a series of numbers entered by the user. Stop accepting numbers once user enters -1 then display the total.

INPUT	PROCESS	OUTPUT
number	repeat input and calculate total until -1 entered	total

```
START
   INPUT num
   SET total = 0
   WHILE num NOT equals to -1
         total = total + num
         INPUT num
   END WHILE
   PRINT total
END
```

Initialization

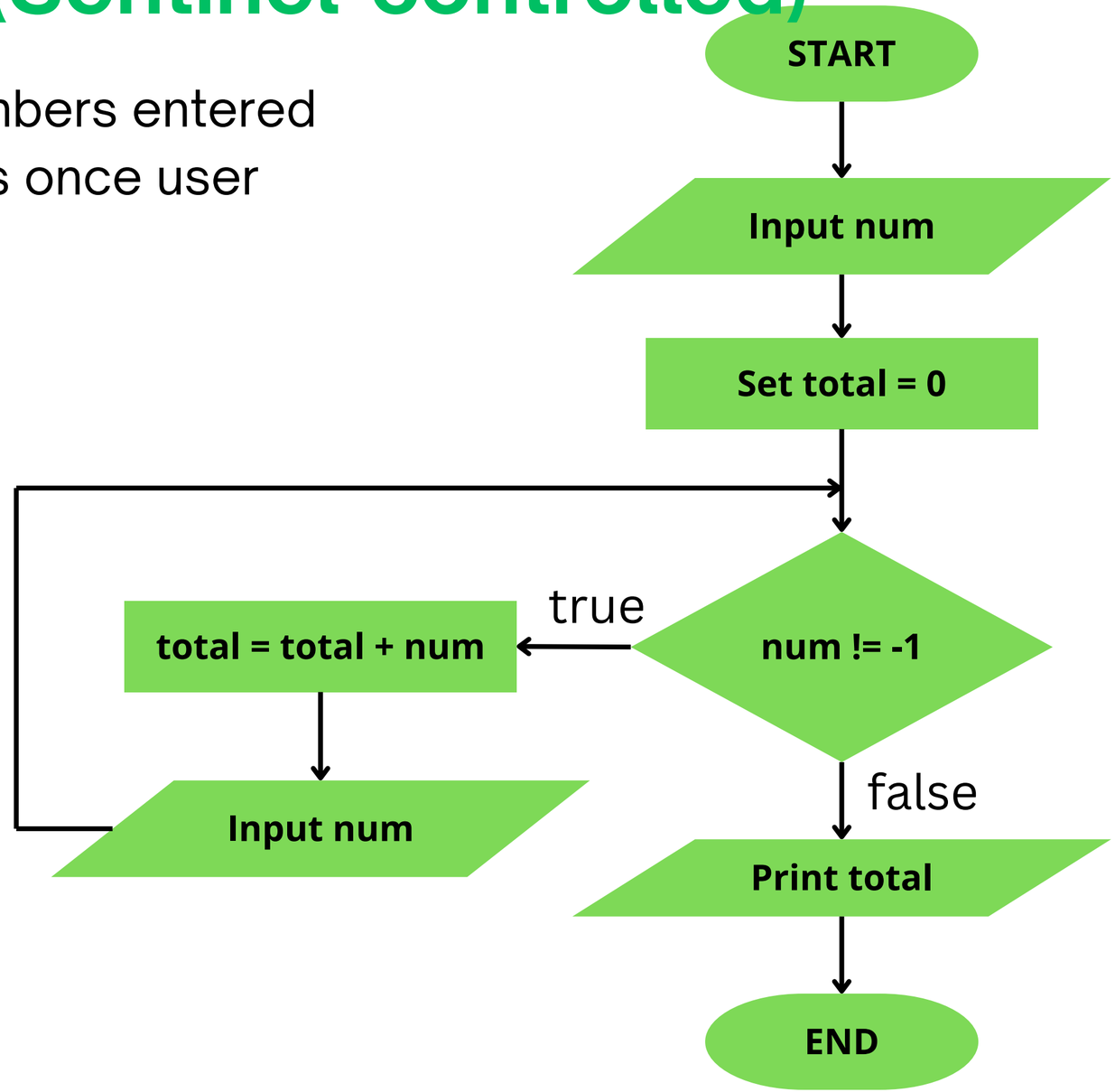
Condition

Update

Example 3 (Sentinel-controlled)

Calculate the total of a series of numbers entered by the user. Stop accepting numbers once user enters -1 then display the total.

```
START
   INPUT num
   SET total = 0
   WHILE num NOT equals to -1
         total = total + num
         INPUT num
   END WHILE
   PRINT total
END
```



Practice Questions

Do problem analysis, then design the solution (pseudocode & flowchart) for the problem statement(s) below.

1. Display the number of times the user entered the number 8.
   The program will keep counting until the number 0 is entered.

Tracing Outputs

Trace Tables

Desk Checking Algorithms

What is Desk Checking?

It is a way of testing an algorithm's logic by manually walking through every line to look for errors and verify that the algorithm works as intended.

A trace table is used to track the variables in an algorithm.

- It allows programmers to predict step-by-step how the computer will run the algorithm.
- It traces the values of variables as each line of code is executed.

Example

This program calculates the total of marks of some students. The number of students, n, will be entered by the user.

INPUT	PROCESS	OUTPUT
n, marks	repeat input marks and calculate total marks until n reached	total marks

Example

This program calculates the total of marks of some students. The number of students, n, will be entered by the user.

```
Start
    total = 0
    counter = 0
    Read n
    while (counter < n)
        Read marks
        total = total + marks
        counter = counter + 1
    endwhile
    Print total
End
```

Example

This program calculates the total of marks of some students. The number of students, n, will be entered by the user. Determine the output of the program if user enters inputs 3, 15, 30, and 64.

total	counter	n	counter < n	marks	total = total + marks	counter = counter + 1	total
0	0	3	true	15	15	1	
			true	30	45	2	
			true	64	109	3	
			false				109

The output of the program is 109.

Exercise

Study the following algorithm then create its trace table and determine the output.

